

$$\varepsilon = \varepsilon_{\text{corpo}}$$

esse parâmetro $\varepsilon_{\text{corpo negro}}$ é chamado de emissividade, e o poder emissivo do corpo real pode ser escrito por

$$E(T) = \varepsilon \sigma T^4$$

Δ 2h

■ Nota Complementar 9.4: método numérico

Na seção 8.6 aparece uma equação cuja incógnita não pode ser encontrada através do álgebra elementar.

$$5e^n - 5 - ne^n = 0$$

Tomando a função f como

$$f(n) = 5e^n - 5 - ne^n$$

Desenhando o seu gráfico é possível ver que a função vale 0 em $n=0$ e $n \approx 4,5$, o que não tem significado físico, em por volta de $n \approx 5$.

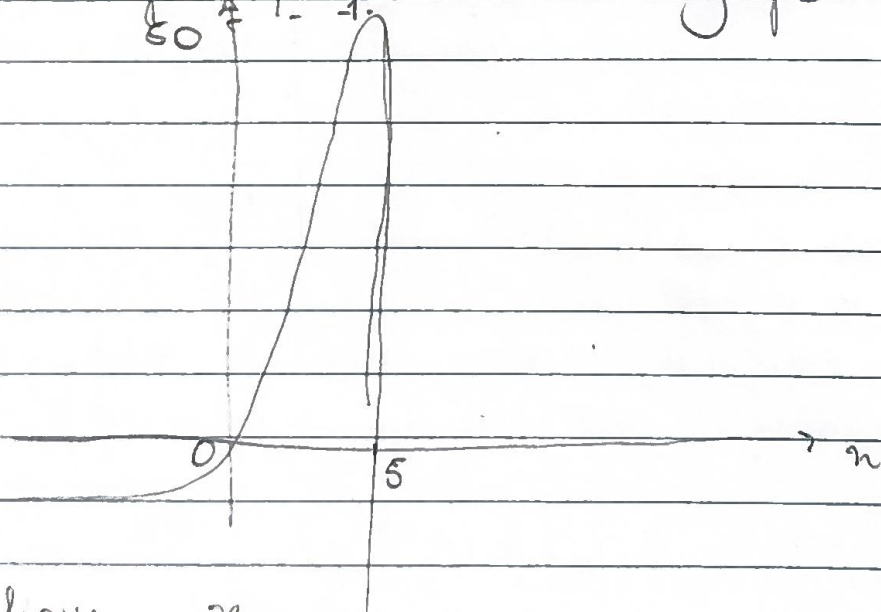


figura 2

Sabemos portanto que a solução que procuramos está de certeza no intervalo $n \in [4,5; 5,5]$. Se for do nosso interesse uma aproximação mais precisa, podemos desenvolver um código. Aqui vou usar a linguagem C. Começamos por importar as bibliotecas standard e matemática e definir a nossa função f .

```
#include <stdio.h>
```

```
#include <math.h>
```

```
double f(double x) {  
    return (5.0 * exp(x) - 5.0 - x * exp(x));  
}
```

Comecemos agora por criar a função main, definindo as variáveis que vamos usar, os extremos do intervalo, a tolerância e a variável para guardar o resultado

```
int main() {  
    double ext_inf = 4.5;  
    double ext_sup = 5.5;  
    double tolerancia = 0.00001;  
    double aprox_result;  
}
```

O algoritmo será o seguinte: enquanto o nosso intervalo for maior que a tolerância achamos o meio do intervalo e aplicamos-lhe a função f . Como para valores inferiores a x_{real} , ($f(x_{real}) = 0$), no intervalo, a função toma valores positivos e superiores a x_{real} valores negativos, se $f(x_{meio}) > 0$, então x_{meio} passa a ser o nosso limite inferior e se $f(x_{meio}) < 0$ x_{meio} passa a ser o limite superior, ver imagem n.



$f(\text{meio}) < 0$

$f(\text{meio}) > 0$

Desta maneira podemos ir reduzindo o intervalo controlado pela tolerância. Quanto menor a tolerância mais exato o valor e mais poder computacional será necessário. Vamos traduzir o algoritmo em C.

```
while (ext_sup - ext_inf > tolerancia) {  
    aprox_result = (ext_sup + ext_inf) / 2;
```

```
    if (f(aprox_result) > 0) {  
        ext_inf = aprox_result;
```

```
    }  
    else ext_sup = aprox_result;
```

```
}  
aprox-result = (ext-sup + ext-inf)/2;  
printf("n = %0.5f\n", aprox-result);  
printf("f(n) = %0.7f\n", f(aprox-result));  
}
```

Assim, o código total fica

pergunta

e ao executar achamos $n = 4,96511 \pm 0,00001$ e $f(n) = -0,0000500$, bastante próximo de 0.